

Module-26 React with TypeScript Homework

What is React? React vs Next.js

Learning Goal: Understand the difference between a UI library and a framework.

1. In your own words, write a short paragraph (5–7 sentences) explaining what React is and what problem it solves.
2. Create a table (on paper or in a text file) listing at least 4 differences between React and Next.js.
3. Research and name 2 real-world companies/products that use React, and 2 that use Next.js.
4. Answer: Would you use plain React or Next.js for a blog that needs strong SEO? Justify your answer in 2–3 sentences.

Components and Component Types

Learning Goal: Practice creating and organizing basic React components.

1. Define, in your own words, what a component is?
2. Create a new functional component called `WelcomeCard.tsx` that returns a div containing a heading and a paragraph.
3. Import and render your `WelcomeCard` component inside `App.tsx`.
4. Create a second component called `Footer.tsx` and render it below `WelcomeCard` in `App.tsx`.

JSX with TypeScript — Rules and Dynamic Content

Learning Goal: Get comfortable writing valid JSX and embedding dynamic values.

1. List 4 JSX rules you must follow when writing `.tsx` files (e.g. `className`, single root element).
2. Create a component called `ProfileCard.tsx` with variables for your name, age, and favorite hobby (typed with TypeScript).
3. Display all three variables inside JSX using curly braces `{ }`.
4. Add one JSX expression that performs a calculation (e.g. birth year based on current year and age) and display the result.
5. Identify and fix the JSX error in this snippet:

```
<div><h1>Hello</h1><p>Welcome</p></div><p>Extra</p>
```

Exploring JSX and Props Basics

Learning Goal: Understand what props are and why components use them.

1. In your own words, explain what a 'prop' is and why props make components reusable.
2. List 3 rules about how props behave in React (e.g. read-only, one-directional flow).
3. Sketch how data flows from a parent App component to a child ProductCard component.

Passing and Reading Props

Learning Goal: Practice sending data into a component and reading it back out.

1. Create a ProductCard.tsx component that accepts productName (string) and price (number) as typed props.
2. In App.tsx, render at least 3 ProductCard components, each with different prop values.
3. Rewrite your ProductCard component two ways: once reading props.propertyName, and once using destructuring in the function signature.
4. Add a 4th prop called inStock (boolean) and display 'Available' or 'Out of Stock' based on its value.

Read-Only Props and Intro to Conditional Rendering

Learning Goal: Understand prop immutability and write your first conditional UI.

1. Explain in 2–3 sentences why props should never be modified inside a child component.
2. Identify the bug in this code and explain why it is incorrect:

```
function Card({title}: {title:string})  
{  
  title = 'New Title';  
  return <h2>{title}</h2>; }  
}
```

3. Create a component StatusBadge.tsx that accepts an isActive boolean prop and uses an if/else statement to return different JSX depending on its value.

Conditional Rendering — If, Ternary, AND/OR

Learning Goal: Practice all three conditional rendering techniques and know when to use each.

1. Rewrite your StatusBadge component from Task 26-6 using the ternary operator instead of if/else.
2. Create a component Cart.tsx that accepts an itemCount (number) prop and uses && to show a 'Checkout' button only when itemCount is greater than 0.
3. Create a component UserGreeting.tsx that accepts an optional username prop and uses || to display 'Guest' when no username is provided.

4. Write a short comparison (3–4 sentences) on when you would choose if/else vs ternary vs `&&/||` in a real project.

Rendering Lists Using `map()`

Learning Goal: Practice transforming array data into a list of rendered components.

1. Create a TypeScript interface called `Student` with `id` (number), `name` (string), and `grade` (number).
2. Create an array of at least 5 `Student` objects.
3. Create a `StudentList.tsx` component that uses `.map()` to render each student's name and grade inside a `<ul><li>` list.
4. Make sure each rendered `<li>` has a unique `key` prop using the student's `id` (not the array index).
5. Bonus: Use conditional rendering inside the `map()` to display 'Pass' if `grade >= 40`, otherwise 'Fail'.

Mini Project

Learning Goal: Consolidate everything learned across the module into one working mini-project.

1. Build a single `App.tsx` that combines everything: a title, a `UserList` rendered from an array using `map()`, each user card receiving props, and conditional rendering to show an 'Active' or 'Inactive' badge for each user.
2. Write a short reflection (5–8 sentences) on what you found most challenging in this module and what you understood well.

